My Project

Generated by Doxygen 1.13.2

Chapter 1

README

Ši programa leidžia:

Įvesti studentų duomenis rankiniu būdu, automatiškai arba iš failo Apskaičiuoti galutinį pažymių rezultatą pagal vidurkį arba medianą Skirstyti studentus į dvi grupes: pažangius ir nepažangius pagal dvi strategijas Rezultatus išvesti į ekraną arba į failus alfos.txt ir susmukeliai.txt Rūšiuoti studentus pagal vardą, pavardę arba galutinį rezultatą

Programa naudoja dvi klases. Bazinę Žmogaus klasę, kuri aprašo bendrus žmogaus atributus (vardas, pavardė). Turi gryną virtualią funckiją test(), todėl iš jos negalima kurti objektų. Ir klasę Studentas, kuri yra paveldima iš Žmogaus klasės. Joje saugomi Studento pažymiai ir galutinis rezultatas. Implementuoja test() metodą, galima kurti Studento tipo objektus.

Perdengtų metodų paaiškinimas

---------- Įvesties būdai ----------

Tipas	Realizacija	Aprašymas
Rankinis	pasirinkimas == 1	Vartotojas suveda vardą, pavardę, ND ir egzaminą
Pusiau auto	pasirinkimas == 2	Automatiškai generuojami tik pažymiai
Automatinis	pasirinkimas == 3	Visi duomenys sugeneruojami automatiškai
Iš failo	failo pasirinkimas == 1,2,3,4,5	Visi duomenys sugeneruojami kursiokai.txt faile

---------- Išvesties būdai ----------

Tipas	Realizacija	Failai
Ekranas	pasirinkimas == 1, spausdinam()	Duomenys išvedami į ekraną pagal rikiavimą
Į failą	pasirinkimas == 2, spausdinam()	Sukuriami alfos.txt ir susmukeliai.txt

==== REZULTATŲ LENTELĖ ====

Veiksmas	V1.0 (1M)	V1.1 (1M)	V1.0 (10M)	V1.1 (10M)
Failo kūrimo laikas	3.15s	3.02s	29.61s	29.57s
Duomenų nuskaitymo laikas	7.54s	8.08s	76.73s	82.71s
Skirstymo laikas	0.84s	1.14s	9.70s	12.18s
Rezultatų išvedimo laikas	0.40s	0.39s	3.96s	4.25s
Rūšiavimo tvarka laikas	0.04s	0.18s	0.45s	1.69s
----------——	--------—	---------—	----------—	----------—
Visos programos laikas	18.68s	21.86s	136.59s	146.66s

==== FLAGŲ PALYGINIMAS ====

Veiksmas	V1 -O1	V1 -O2	V1 -O3	V1.1 -O1	V1.1 -O2	V1.1 -O3
Failo kūrimo laikas	2.96s	2.99s	3.00s	2.97s	3.01s	2.97s
Duomenų nuskaitymo laikas	7.48s	7.51s	7.53s	8.09s	8.09s	8.09s
Skirstymo į blogus/gerus laikas	0.84s	0.83s	0.83s	1.14s	1.15s	1.15s
Rezultatų išvedimo laikas	0.40s	0.40s	0.40s	0.42s	0.40s	0.45s
Rūšiavimo tvarka laikas	0.03s	0.03s	0.04s	0.18s	0.18s	0.18s
----------——	-------—	-------—	-------—	---------—	---------—	---------—
Visos programos laikas	19.06s	19.32s	18.03s	18.01s	17.16s	17.05s

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 4

File Index

4.1 File List

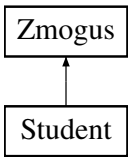
Here is a list of all documented files with brief descriptions:

Chapter 5

Class Documentation

5.1 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- void test ()
- **Student** (const Student &other)
- Student & **operator=** (const Student &other)
- **Student** (Student &&other)
- Student & **operator=** (Student &&other)
- void **setVid** (double Vid)
- void **setEgz** (int Egz)
- void **setNd** (vector< int > &Nd)
- double **getVid** () const
- int **getEgz** () const
- vector< int > & **getNd** ()

Public Member Functions inherited from Zmogus

- void **setName** (string &Name)
- void **setSurn** (string &Surn)
- string **getName** () const
- string **getSurn** () const

Friends

- istream & **operator>>** (std::istream &in, Student &s)
- ostream & **operator<<** (ostream &os, const Student &s)

Additional Inherited Members

Protected Attributes inherited from Zmogus

- string **name**
- string **surn**

5.1.1 Member Function Documentation

5.1.1.1 test()

```
void Student::test ()  [inline], [virtual]
```

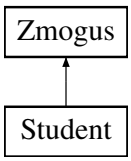
Implements Zmogus.

The documentation for this class was generated from the following file:

- V1.5vect.h

5.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- void **setName** (string &Name)
- void **setSurn** (string &Surn)
- string **getName** () const
- string **getSurn** () const
- virtual void **test** ()=0

Protected Attributes

- string **name**
- string **surn**

The documentation for this class was generated from the following file:

- V1.5vect.h

Chapter 6

File Documentation

6.1 lib.h

```
00001 #ifndef LIB_H
00002 #define LIB_H
00003 #include <iostream>
00004 #include <iomanip>
00005 #include <vector>
00006 #include <chrono>
00007 #include <ctime>
00008 #include <fstream>
00009 #include <sstream>
00010 #include <time.h>
00011 #include <random>
00012 #include <list>
00013 #include <deque>
00014 #include <algorithm>
00015 #include <utility>
00016
00017 using std::string;
00018 using std::vector;
00019 using std::cout;
00020 using std::cin;
00021 using std::endl;
00022 using std::ifstream;
00023 using std::istringstream;
00024 using std::ofstream;
00025 using std::setw;
00026 using std::left;
00027 using std::fixed;
00028 using std::setprecision;
00029 using std::sort;
00030 using std::swap;
00031 using std::numeric_limits;
00032 using std::streamsize;
00033 using std::ostringstream;
00034 using std::list;
00035 using std::deque;
00036 using std::copy;
00037 using std::ostream;
00038 using std::istream;
00039
00040
00041 #endif
```

6.2 V1.5vect.h

```
00001 #include "lib.h"
00002
00003 //random skaiciu generavimas
00004 using hrClock = std::chrono::high_resolution_clock;
00005 std::mt19937 mt(static_cast<long unsigned int>(hrClock::now().time_since_epoch().count()));
00006 std::uniform_int_distribution<int> dist(1, 10);
00007
00008
00009
```

```
00010 class Zmogus{
00011     protected:
00012         string name; // vardas
00013         string surn; // pavarde
00014     public:
00015         Zmogus() : name(""), surn("") {} // konstruktorius
00016         virtual ~Zmogus(){}
00017 
00018         void setName(string& Name) {
00019             name = Name;
00020         }
00021 
00022         void setSurn(string& Surn) {
00023             surn = Surn;
00024         }
00025 
00026         string getName() const {
00027             return name;
00028         }
00029 
00030         string getSurn() const {
00031             return surn;
00032         }
00033         virtual void test() = 0;
00034 };
00035 
00036 
00037 class Student : public Zmogus {
00038     private:
00039         vector<int> nd; // nd rezultatai
00040         int egz; // egzaminu rez
00041         double vid; // galutinis vidurkis
00042 
00043     public:
00044 
00045         void test(){}
00046 
00047         Student() : egz(0), vid(0) {} // konstruktorius
00048 
00049         ~Student(){} // destruktorius
00050 
00051         // copy constructor dada
00052         Student(const Student& other)
00053         : Zmogus(other), nd(other.nd), egz(other.egz), vid(other.vid){}
00054 
00055         // copy assignment operator
00056         Student& operator=(const Student& other){
00057             if(this != &other) {
00058                 name = other.name;
00059                 surn = other.surn;
00060                 nd = other.nd;
00061                 egz = other.egz;
00062                 vid = other.vid;
00063             }
00064             return *this;
00065         }
00066 
00067        // move constructor
00068         Student(Student&& other)
00069             : Zmogus(std::move(other)),
00070             nd(std::move(other.nd)),
00071             egz(std::move(other.egz)),
00072             vid(std::move(other.vid))
00073         {
00074             other.egz = 0;
00075             other.vid = 0.0;
00076             other.name = "";
00077             other.surn = "";
00078             other.nd.clear();
00079         }
00080 
00081         // move assignment operator
00082         Student& operator=(Student&& other) {
00083             if (this != &other) {
00084                 name = std::move(other.name);
00085                 surn = std::move(other.surn);
00086                 nd = std::move(other.nd);
00087                 egz = std::move(other.egz);
00088                 vid = std::move(other.vid);
00089 
00090                 other.egz = 0;
00091                 other.vid = 0.0;
00092                 other.name = "";
00093                 other.surn = "";
00094                 other.nd.clear();
00095             }
00096             return *this;
```

```
00097         }
00098
00099         //seteriai
00100         void setVid(double Vid) {
00101             vid = Vid;
00102         }
00103
00104         void setEgz(int Egz) {
00105             egz = Egz;
00106         }
00107
00108         void setNd(vector<int>& Nd) {
00109             nd = Nd;
00110         }
00111
00112
00113         //geteriai
00114         double getVid() const {
00115             return vid;
00116         }
00117
00118         int getEgz() const {
00119             return egz;
00120         }
00121
00122         vector<int>& getNd(){
00123             return nd;
00124         }
00125
00126         friend istream& operator»(std::istream& in, Student& s) {
00127             std::string name, surname;
00128             std::vector<int> nd;
00129             int temp, egz;
00130
00131             in » name » surname;
00132             s.setName(name);
00133             s.setSurn(surname);
00134
00135             while (in » temp) {
00136                 nd.push_back(temp);
00137             }
00138
00139             if (!nd.empty()) {
00140                 egz = nd.back();
00141                 nd.pop_back();  // Remove last element, which is egz
00142                 s.setEgz(egz);
00143                 s.setNd(nd);
00144             }
00145
00146             in.clear();  // Clear stream state if end of line or error
00147             return in;
00148         }
00149
00150         // isvesties operatorius
00151         friend ostream& operator«(ostream& os, const Student& s){
00152             os « left « setw(20) « s.name « setw(16) « s.surn « s.vid « endl;
00153             return os;
00154         }
00155 };
00156
00157 std::chrono::duration<double> generationTime; // generavimo laikas
00158 std::chrono::duration<double> readTime; // skaitymo laikas
00159 std::chrono::duration<double> sortTime; // skirstymo laikas
00160 std::chrono::duration<double> writeTime; // rasymo laikas
00161 std::chrono::duration<double> rusiavimoLaikas; // rusiavimo laikas
00162
00163 vector<Student> BadStudents;
00164 vector<Student> GoodStudents;
00165
00166 void skaitom(int pasirinkimas, string A[], string B[]);
00167 void vidurkis();
00168 void mediana();
00169 void spausdinam(char a);
00170 void generuojam(string b, int n);
00171 void rusiuojam2(); // skaidymas per puse
00172 void rusiuojam1(); // skaidymas is vieno konteinerio i du
00173
00174 bool compareByName(const Student& a, const Student& b) {
00175     return a.getName() < b.getName();
00176 }
00177 bool compareBySurname(const Student& a, const Student& b) {
00178     return a.getSurn() < b.getSurn();
00179 }
00180 bool compareByVid(const Student& a, const Student& b) {
00181     return a.getVid() < b.getVid();
00182 }
00183
```

```
00184 void testas() {
00185
00186     //Zmogus zmogus; //testas kad negalima sukurti zmogaus objekto
00187
00188     string v = "Jonas";
00189     string p = "Jonaitis";
00190     Student s1;
00191     s1.setName(v);
00192     s1.setSurn(p);
00193     s1.setEgz(8);
00194     vector<int> nd = {7, 8, 9};
00195     s1.setNd(nd);
00196     s1.setVid(8.2);
00197
00198     // Test copy constructor
00199     Student s2(s1);
00200     cout « "Student 1:\n" « s1;
00201     cout « "Student 2 (copied from Student 1):\n" « s2;
00202
00203     // Test copy assignment operator
00204     Student s3;
00205     s3 = s1;
00206     cout « "Student 1:\n" « s1;
00207     cout « "Student 3 (assigned from Student 1):\n" « s3;
00208
00209     // Test move constructor
00210     cout « "Student 1 before:\n" « s1;
00211     Student s4(std::move(s1));
00212     cout « "Student 1 after:\n" « s1;
00213     cout « "Student 4 (moved from Student 1):\n" « s4;
00214
00215
00216     // Test move assignment operator
00217     Student s5;
00218     cout « "Student 2 before:\n" « s2;
00219     s5 = std::move(s2);
00220     cout « "Student 2 after:\n" « s2;
00221     cout « "Student 5 (moved from Student 2):\n" « s5;
00222 }
00223
00224
00225 void rusiuojam1(){
00226     // nukopijuojam visus elementus i atskira konteineri, kad nereiktu keist toliau esancios programos
00227     vector<Student> BadStudents2(BadStudents);
00228     BadStudents.clear();
00229
00230     auto startSort = std::chrono::high_resolution_clock::now();
00231     sort(BadStudents2.begin(), BadStudents2.end(), compareByVid);
00232     while(!BadStudents2.empty()){
00233         if(BadStudents2.back().getVid() >= 5){
00234             GoodStudents.push_back(std::move(BadStudents2.back()));
00235         } else {
00236             BadStudents.push_back(std::move(BadStudents2.back()));
00237         }
00238         BadStudents2.pop_back(); // istrinam paskutini studenta
00239     }
00240
00241     auto endSort = std::chrono::high_resolution_clock::now();
00242     sortTime = endSort - startSort;
00243 }
00244
00245 void rusiuojam2(){
00246     // surusiuojam studentus pagal galutini bala
00247     sort(BadStudents.begin(), BadStudents.end(), compareByVid);
00248         auto startSort = std::chrono::high_resolution_clock::now();
00249         // iteruojam nuo galo
00250         while(BadStudents.back().getVid() >= 5) {
00251             GoodStudents.push_back(std::move(BadStudents.back()));
00252             BadStudents.pop_back(); // istrinam paskutini studenta
00253         }
00254         auto endSort = std::chrono::high_resolution_clock::now();
00255         sortTime = endSort - startSort;
00256 }
00257
00258 void skaitom(int pasirinkimas, string A[], string B[]){
00259     bool testi = true;
00260     int i = 0;
00261     string name, surn;
00262     int egz;
00263     vector <int> ND;
00264     Student temp;
00265     while(testi){
00266         char teesti;
00267
00268         //mokinio vardas pavarde
00269         if(pasirinkimas == 1 || pasirinkimas == 2){
00270             cout « "įveskite mokinio vardą: ";
```

```
00271             cin » name;
00272             cout « "įveskite mokinio pavardę: ";
00273             cin » surn;
00274         }else if(pasirinkimas == 3){
00275             name = A[dist(mt)];
00276             surn = B[dist(mt)];
00277         }
00278 
00279         //nd rezultatai
00280         bool testi2 = true;
00281         int j = 0;
00282         while(testi2){
00283             char teesti2;
00284             int nd_result;
00285             if(pasirinkimas == 1){
00286                 while (true){
00287                     try{
00288                         cout « "įveskite " « j+1 « " namų darbo rezultatą: ";
00289                         cin » nd_result;
00290                         if(nd_result < 0 || nd_result > 10){
00291                             throw std::invalid_argument ("klaida, įveskite skaičių nuo 0 iki 10");
00292                         }
00293                         break;
00294                     }
00295                     catch(const std::invalid_argument& e){
00296                     cout « e.what() « endl;
00297                     cin.clear();
00298                     cin.ignore(123, '\n');
00299                     }
00300                 }
00301             }else if(pasirinkimas == 2 || pasirinkimas == 3) ND.push_back(dist(mt));
00302 
00303             while(true){
00304                 try{
00305                     cout « "ar norite pridėti daugiau namų darbų rezultatų? (t/n): ";
00306                     cin » teesti2;
00307                     if(!(teesti2 == 't' || teesti2 == 'n')){
00308                         throw std::invalid_argument("klaida, pasirinkite taip(t) arba ne(n)");
00309                     }
00310                     break;
00311                 }
00312                 catch(const std::invalid_argument& e){
00313                     cout « e.what() « endl;
00314                     cin.clear();
00315                     cin.ignore(123, '\n');
00316                 }
00317             }
00318 
00319             if(teesti2 == 'n'){
00320                 testi2 = false;
00321             } else testi2 = true;
00322             j++;
00323             if(j == 20) break;
00324         }
00325 
00326 
00327         //egzamino rezultatas
00328         if(pasirinkimas == 1){
00329             while (true){
00330                 try{
00331                     cout « "įveskite egzamino rezultatą: ";
00332                     cin » egz;
00333                     if(egz < 0 || egz > 10){
00334                         throw std::invalid_argument ("klaida, įveskite skaičių nuo 0 iki 10");
00335                     }
00336                     break;
00337                 }
00338                 catch(const std::invalid_argument& e){
00339                     cout « e.what() « endl;
00340                     cin.clear();
00341                     cin.ignore(123, '\n');
00342                 }
00343             }
00344         }else if(pasirinkimas == 2 || pasirinkimas == 3) egz = dist(mt);
00345 
00346         temp.setName(name); // Set the name
00347         temp.setSurn(surn); // Set the surname
00348         temp.setEgz(egz);   // Set the exam result
00349         temp.setNd(ND);     // Set the homework results (ND is a vector<int>)
00350 
00351         // Add the Student object to the BadStudents vector
00352         BadStudents.push_back(temp);
00353 
00354         while(true){
00355             try{
00356                 cout « "ar norite pridėti daugiau mokinių? (t/n): ";
00357                 cin » teesti;
```

```
00358                 if(!(teesti == 't' || teesti == 'n')){
00359                     throw std::invalid_argument("klaida, pasirinkite taip(t) arba ne(n)");
00360                 }
00361                 break;
00362             }
00363             catch(const std::invalid_argument& e){
00364                 cout << e.what() << endl;
00365                 cin.clear();
00366                 cin.ignore(123, '\n');
00367             }
00368         }
00369 
00370         if(teesti == 'n'){
00371             testi = false;
00372         }
00373         i++;
00374         if(i == 15) break;
00375     }
00376 }
00377 
00378 
00379 void generuojam(string b, int n){
00380 
00381     ostringstream oss;
00382     oss << left << setw(20) << "Vardas" << setw(20) << "Pavardė";
00383     for(int i = 1; i <= 15; i++){
00384         oss << "ND" << setw(5) << i;
00385     }
00386     oss << "Egz." << endl;
00387 
00388     for(int i = 0; i < n; i++){
00389         oss << left << "Vardas" << setw(14) << i+1 << "Pavardė" << setw(12) << i+1;
00390         for(int j = 0; j < 15; j++){
00391             oss << setw(7) << dist(mt);
00392         }
00393         oss << dist(mt)<< endl;
00394     }
00395     ofstream fr(b);
00396     fr << oss.str();
00397     fr.close();  // Close file
00398 }
00399 
00400 
00401 void vidurkis(){
00402     for(int i = 0; i < BadStudents.size(); i++){
00403         double sum = 0;
00404         for(int j = 0; j < BadStudents[i].getNd().size(); j++){
00405             sum += BadStudents[i].getNd()[j];
00406         }
00407         double average;
00408         average = (sum / BadStudents[i].getNd().size())*0.4 + (BadStudents[i].getEgz()*0.6);
00409         BadStudents[i].setVid(average);
00410     }
00411 }
00412 
00413 
00414 void mediana(){
00415 
00416     //nd rezultatu rikiavimas didejimo tvarka
00417     for (int i = 0; i < BadStudents.size(); i++) {
00418         sort(BadStudents[i].getNd().begin(), BadStudents[i].getNd().end());
00419     }
00420 
00421     //medianos skaiciavimas
00422     for(int i = 0; i < BadStudents.size(); i++){
00423         double average;
00424         if(BadStudents[i].getNd().size() % 2 == 0){
00425             average = ((BadStudents[i].getNd()[BadStudents[i].getNd().size()/2] +
      BadStudents[i].getNd()[BadStudents[i].getNd().size()/2 - 1]) / 2.0)*0.4 +
      (BadStudents[i].getEgz()*0.6);
00426             BadStudents[i].setVid(average);
00427         }
00428         else {
00429             average = BadStudents[i].getNd()[BadStudents[i].getNd().size()/2]*0.4 +
      (BadStudents[i].getEgz()*0.6);
00430             BadStudents[i].setVid(average);
00431         }
00432     }
00433 }
00434 
00435 
00436 void spausdinam(char a) {
00437 
00438     int pasirinkimas;
00439     while(true){
00440         try{
00441             cout << "Kur norite matyti rezultatus?" << endl;
```

```
00442             cout « "1 - ekrane" « endl;
00443             cout « "2 - faile" « endl;
00444             cin » pasirinkimas;
00445             if(pasirinkimas != 1 && pasirinkimas != 2){
00446                 throw std::invalid_argument("klaida, įveskite skaičių 1 arba 2");
00447             }
00448             break;
00449         }
00450         catch(const std::invalid_argument& e){
00451             cout « e.what() « endl;
00452             cin.clear();
00453             cin.ignore(123, '\n');
00454         }
00455     }
00456
00457     int rusiavimas;
00458     while(true){
00459         try{
00460             cout « "Kaip norite surūšiuoti rezultatus?" « endl;
00461             cout « "1 - pagal vardą" « endl;
00462             cout « "2 - pagal pavardę" « endl;
00463             cout « "3 - pagal galutinį balą" « endl;
00464             cin » rusiavimas;
00465             if(rusiavimas != 1 && rusiavimas != 2 && rusiavimas != 3){
00466                 throw std::invalid_argument("klaida, įveskite skaičių 1, 2 arba 3");
00467             }
00468             break;
00469         }
00470         catch(const std::invalid_argument& e){
00471             cout « e.what() « endl;
00472             cin.clear();
00473             cin.ignore(123, '\n');
00474         }
00475     }
00476
00477     auto startRusiavimas = std::chrono::high_resolution_clock::now();
00478     if(rusiavimas == 1){
00479         sort(BadStudents.begin(), BadStudents.end(), compareByName);
00480         sort(GoodStudents.begin(), GoodStudents.end(), compareByName);
00481     } else if(rusiavimas == 2){
00482         sort(BadStudents.begin(), BadStudents.end(), compareBySurname);
00483         sort(GoodStudents.begin(), GoodStudents.end(), compareBySurname);
00484     } else{
00485         sort(BadStudents.begin(), BadStudents.end(), compareByVid);
00486         sort(GoodStudents.begin(), GoodStudents.end(), compareByVid);
00487     }
00488     auto endRusiavimas = std::chrono::high_resolution_clock::now();
00489     rusiavimoLaikas = endRusiavimas - startRusiavimas;
00490
00491     if(pasirinkimas == 1){
00492         cout « left « setw(20) « "Pavardė" « setw(15) « "Vardas" « setw(20);
00493
00494         if (a == 'v') {
00495             cout « "Galutinis (Vid.)" « endl;
00496         } else {
00497             cout « "Galutinis (Med.)" « endl;
00498         }
00499
00500         cout « "-------------------------------------------------------------" « endl;
00501
00502         cout « fixed « setprecision(2);
00503
00504         for (const auto& student : BadStudents) {
00505             cout « student « endl;
00506         }
00507         for (const auto& student : GoodStudents) {
00508             cout « student « endl;
00509         }
00510     } else {
00511
00512         // geri mokiniai
00513
00514         auto startWrite = std::chrono::high_resolution_clock::now();
00515         ostringstream oss;
00516         oss « left « setw(20) « "Vardas" « setw(15) « "Pavardė" « setw(20);
00517
00518         if (a == 'v') {
00519             oss « "Galutinis (Vid.)" « endl;
00520         } else {
00521             oss « "Galutinis (Med.)" « endl;
00522         }
00523
00524         oss « "-------------------------------------------------------------" « endl;
00525         oss « fixed « setprecision(2);
00526         for (const auto& student : BadStudents) {
00527             oss « student;
00528         }
```

```
00529         ofstream file2("susmukeliai.txt");
00530         file2 « oss.str();
00531         file2.close();
00532 
00533         oss.str("");
00534         oss.clear();
00535 
00536         //blogi mokiniai
00537         oss « left « setw(20) « "Vardas" « setw(15) « "Pavardė" « setw(20);
00538 
00539         if (a == 'v') {
00540             oss « "Galutinis (Vid.)" « endl;
00541         } else {
00542             oss « "Galutinis (Med.)" « endl;
00543         }
00544 
00545         oss « "-------------------------------------------------------------" « endl;
00546 
00547         oss « fixed « setprecision(2);
00548         for (const auto& student : GoodStudents) {
00549             oss « student;
00550         }
00551         ofstream file3("alfos.txt");
00552         file3 « oss.str();
00553         file3.close();
00554         auto endWrite = std::chrono::high_resolution_clock::now();
00555         writeTime = endWrite - startWrite;
00556     }
00557 }
```

Index